# Ejercicio 1 — Una computadora mil veces más rápida

**Dificultad:** ★ **Conferencias:** C1 (Eficiencia y modelos de cómputo)

## Enunciado

> *¿Cuánto crece el tamaño de entrada que se resuelve en una hora con un algoritmo $\Theta(n^2)$? ¿Y con uno $\Theta(n \log n)$? Estima los dos factores.*

Es una generalización directa del ejercicio 4 de la conferencia 1 (que pedía lo mismo para $n \log_2 n$ con una mejora de 100×) y del experimento de la sección 7 de esa misma conferencia, que ya trae la tabla con los números (y da $\sqrt{1000} \approx 32$ para $n^2$ sin derivarlo) — pero no la fórmula general ni el caso $n \log n$. Ese es el trabajo real de este ejercicio: no leer la tabla, sino **derivar** por qué las dos columnas se comportan tan distinto.

## 1 Planteamiento general

Fijamos una computadora que ejecuta $R$ operaciones por segundo (modelo RAM de C1, §4: cada instrucción cuesta 1) y le damos un presupuesto de tiempo $T$ = una hora. El presupuesto en operaciones es

$$P = R \cdot T.$$

Si un algoritmo cuesta $f(n)$ operaciones en el peor caso, el mayor tamaño de entrada que resuelve en ese presupuesto es el mayor $n_0$ tal que $f(n_0) \leq P$, es decir (tratando $f$ como invertible, razonable para las $f$ monótonas crecientes que nos interesan aquí):

$$n_0 = f^{-1}(P).$$

Una computadora $S$ veces más rápida (aquí $S = 1000$) tiene presupuesto $S \cdot P$ en el mismo tiempo $T$. El nuevo tamaño resoluble es $n_1 = f^{-1}(S \cdot P)$, y lo que pide el ejercicio es el **factor de crecimiento** $k = n_1 / n_0$.

La forma de $f$ decide completamente cómo se comporta $k$. Ahí está la diferencia esencial entre las dos columnas.

## 2 Caso ideal $f(n) = cn^2$: el factor es $\sqrt{S}$

Sea $f(n) = c \cdot n^2$. Entonces $n_0 = \sqrt{P/c}$ y $n_1 = \sqrt{S \cdot P/c} = \sqrt{S} \cdot n_0$. Por lo tanto

$$k = n_1 / n_0 = \sqrt{S},$$

**exactamente**, sin aproximar nada, y sin que importe el valor de $c$, de $P$, ni de $n_0$. Esto es una propiedad general de las leyes de potencia: si $f(n) = c \cdot n^p$, entonces $f(n_1) = S \cdot f(n_0)$ da $n_1 = S^{1/p} \cdot n_0$, un factor que es **invariante de escala**: no depende de cuán grande sea $n_0$ para empezar.

La palabra "exactamente" se refiere a este modelo ideal $f(n) = cn^2$. La notación $\Theta(n^2)$ por sí sola solo fija el orden asintótico; una función cualquiera de esa clase puede tener factores constantes, redondeos o términos adicionales que cambien el factor finito, aunque no cambien la conclusión asintótica.

Con $S = 1000$: $k = \sqrt{1000} \approx 31.62$.

*(De paso, esto generaliza el punto que hace C1 §7 con $n^3$, donde la misma cuenta da $k = S^{1/3} \approx 10$ para $S = 1000$.)*

## 3 Caso ideal $f(n) = cn \log_2 n$: el factor depende de $n_0$

Aquí $f$ no es una ley de potencia pura, y la invariancia de escala se rompe. Sea $f(n) = c \cdot n \cdot \log_2 n$. Buscamos $n_1 = k \cdot n_0$ tal que $f(n_1) = S \cdot f(n_0)$:

$$\begin{aligned} k \cdot n_0 \cdot \log_2(k \cdot n_0) &= S \cdot n_0 \cdot \log_2 n_0 \\ k \cdot (\log_2 k + \log_2 n_0) &= S \cdot \log_2 n_0 \\ k &= \frac{S}{1 + \log_2 k / \log_2 n_0}. \end{aligned}$$

Esta ecuación no tiene a $k$ despejado (aparece a ambos lados), pero dice algo importante de inmediato:

- $k < S$ siempre que $k > 1$ (el régimen de interés cuando $S > 1$). La mejora en tamaño de entrada es **estrictamente peor** que la mejora en velocidad.
- $k \to S$ **cuando** $n_0 \to \infty$, porque $\log_2 k$ queda acotado (del orden de $\log_2 S$, una constante) frente a $\log_2 n_0 \to \infty$, así que el término de corrección $\log_2 k / \log_2 n_0 \to 0$. Para entradas astronómicamente grandes, un algoritmo $n \log n$ empieza a comportarse, en este sentido particular, casi como uno lineal.
- Para el $n_0$ que de verdad aparece en el presupuesto de una hora (con $R = 10^9$ op/s, $n_0 \approx 9.86 \times 10^{10}$, ver sección 4), $\log_2 n_0 \approx 36.5$, que no es enorme, así que la corrección sí importa y $k \approx 791$, bastante por debajo de 1000.

### Resolviendo numéricamente

La ecuación $k = S/(1 + \log_2 k / \log_2 n_0)$ se resuelve por iteración de punto fijo: arrancar con una estimación de $k$ y refinar. El código real de esta sección y de la siguiente (Python, ejecutable tal cual) vive aparte, en `verif.py` dentro de esta carpeta; lo que sigue es **pseudocódigo**, pensado solo para que la idea del algoritmo se lea rápido, sin el detalle sintáctico de un lenguaje particular.

```
funcion factor_nlogn(S, n0, iteraciones):
    k ← S                // estimacion inicial: ignorar el termino log
    repetir iteraciones veces:
        k ← S / (1 + log2(k) / log2(n0))
    devolver k
```

La iteración converge en unas pocas vueltas (`verif.py` usa `iteraciones=50` de sobra) porque $\log_2$ es una función que varía muy despacio: un error grande en $k$ produce un error minúsculo en $\log_2 k$.

## 4 Verificación exacta con el programa de la conferencia 1

La sección 7 de C1 ya trae el programa que calcula, por búsqueda binaria, el mayor $n$ resoluble en un presupuesto dado. Lo reutilizamos tal cual (es `mayor_n` de la conferencia) para obtener $n_0$ y $n_1$ de verdad y comparar con la predicción analítica de arriba.

```
funcion mayor_n(costo, presupuesto):
    alto ← 2
    mientras costo(alto) ≤ presupuesto:
        alto ← 2 · alto      // duplicar hasta pasarse
    bajo ← alto / 2
```

```
    mientras bajo + 1 < alto:
        medio ← (bajo + alto) / 2
        si costo(medio) ≤ presupuesto:
            bajo ← medio
        sino:
            alto ← medio
    devolver bajo
```

Para cada costo ($n^2$ y $n \log n$) se calculan por separado $n_0$ = `mayor_n(costo, hora)` y $n_1$ = `mayor_n(costo, 1000 · hora)`. Después se compara $n_1/n_0$ con la predicción teórica de la sección anterior.

Salida real, verificada ejecutando el script:

```
    costo   n0 (actual)   n1 (1000x) factor medido factor teorico
      n^2      1.897e+06        6e+07       31.623        31.623
  n log n      9.857e+10    7.801e+13      791.366       791.366
```

El factor medido con el programa real y el factor que predice la ecuación de punto fijo **coinciden** en los tres decimales impresos (la pequeña diferencia restante viene de que $n_1$ es un entero). Esto es justo la tabla de C1 §7 (columnas "actual" y "1000x", con los mismos órdenes de magnitud: $1.9 \times 10^6 \to 6 \times 10^7$ y $9.86 \times 10^{10} \to 7.8 \times 10^{13}$), pero ahora con la fórmula que explica por qué esos números son esos y no otros, en vez de solo leerlos de una corrida.

**Respuesta a la pregunta del enunciado**

Con una computadora 1000× más rápida, el tamaño de entrada resoluble en una hora crece:

- para el modelo ideal $f(n) = cn^2$ (representante de $\Theta(n^2)$): por un factor $\sqrt{1000} \approx 31.6$ — **exacto**, para cualquier presupuesto de partida;
- para el modelo ideal $f(n) = cn \log n$ (representante de $\Theta(n \log n)$): por un factor $\approx 791$ — **depende** del presupuesto de partida (aquí, una hora a $10^9$ operaciones/segundo); para otro presupuesto el número cambia, siempre por debajo de 1000 y acercándose a 1000 cuanto mayor sea $n_0$ (sección 3).

Mirado al revés — a **tamaño fijo** en vez de a **tiempo fijo** — la respuesta es mucho más simple, y por eso vale la pena contrastarla: si resolver una instancia de tamaño $n_0$ tardaba una hora (60 minutos) con la computadora original, esa **misma** instancia $n_0$ con la computadora 1000× más rápida tarda exactamente $60/1000 = 0.06$ minutos (3.6 segundos) — **sin importar** si el algoritmo es $\Theta(n^2)$, $\Theta(n \log n)$ o cualquier otro, porque a tamaño fijo el tiempo es simplemente proporcional a $1/R$. La pregunta del enunciado es la inversa (a **tiempo** fijo, cuánto crece $n$), y ahí es donde la forma del algoritmo sí importa — esa asimetría entre las dos preguntas es, precisamente, todo el punto de este ejercicio.

## 5 Por qué importa: la asimetría es el punto de la clase

La sección 7 de C1 y su resumen (§11: "una computadora más rápida beneficia mucho a los algoritmos lineales y casi nada a los exponenciales") ya apuntan a esto, y lo que este ejercicio añade es el porqué: el factor de mejora depende de la forma exacta de la función de costo, no solo de su clase de crecimiento. Concretamente, multiplicar la velocidad por 1000 (los modelos ideales $n$, $n^3$ y $2^n$ salen de la misma cuenta y de la tabla de C1 §7; `verif.py` solo mide $n^2$ y $n \log n$):

| Modelo ideal | Factor de crecimiento del tamaño resoluble |
|---|---|
| $f(n) = cn$ | 1000 (exacto) |
| $f(n) = cn \log n$ | $\approx 791$ en el presupuesto de referencia |
| $f(n) = cn^2$ | $\sqrt{1000} \approx 31.6$ |
| $f(n) = cn^3$ | $1000^{1/3} = 10$ |
| $f(n) = c2^n$ | $+\log_2 1000 \approx +10$ (*aditivo*, no multiplicativo) |

Y la sección 7 de C1 todavía da la vuelta adicional que conviene mencionar en cualquier respuesta completa de este ejercicio: si en vez de preguntar "¿cuánto crece lo que puedo resolver?" preguntamos "¿qué pasa si los datos también crecen con los años?", la conclusión se invierte. Si en una década la computadora es 10× más rápida y los datos a procesar son 10× más grandes, un algoritmo lineal tarda exactamente lo mismo que hoy, pero uno cuadrático tarda **10 veces más** (el costo creció ×100, la velocidad solo ×10). Con el tiempo, un algoritmo cuadrático empeora, no mejora — razón de peso para preferir mejorar el algoritmo en vez de esperar al hardware.

## La misma asimetría, ahora acumulada en el tiempo

El ejemplo de arriba es de **una** década. Vale la pena generalizarlo, con la misma herramienta de la sección 2: si cada década el hardware $y$ los datos crecen ambos por un factor $F$, un algoritmo $\Theta(n^p)$ tiene costo $\propto n^p$ y velocidad $\propto F^k$ después de $k$ décadas (con $n$ creciendo a $F^k n_0$), así que su tiempo relativo al de hoy es

$$\frac{\text{tiempo}(k)}{\text{tiempo}(0)} = \frac{(F^k n_0)^p / F^k}{n_0^p} = F^{k(p-1)}.$$

Para $p = 1$ (lineal), el exponente es 0: el tiempo **nunca cambia**, para cualquier $k$ — es la misma cuenta exacta que arriba, ahora para todas las décadas, no solo una. Para $p = 2$ (cuadrático), el exponente es $k$: el tiempo crece como $F^k$, **se multiplica otra vez por $F$ en cada década siguiente** (no es un efecto de una sola vez). Con $F = 10$:

| Década | lineal ($p$=1) | cuadrático ($p$=2) | cúbico ($p$=3) |
|---|---|---|---|
| hoy ($k$=0) | ×1 | ×1 | ×1 |
| +10 años | ×1 | ×10 | ×100 |
| +20 años | ×1 | ×100 | ×10,000 |
| +30 años | ×1 | ×1000 | ×1,000,000 |

**Verificado** calculando $F^{k(p-1)}$ directamente, sin aproximar:

```
decada +anios lineal (p=1) cuadratico (p=2) cubico (p=3)
     0      0            1                1           1
     1     10            1               10         100
     2     20            1              100       10000
     3     30            1             1000     1000000
```

El algoritmo cúbico ni siquiera necesita treinta años para volverse impracticable: en veinte ya cuesta diez mil veces más que hoy. Cuanto mayor el exponente $p$, más rápido un algoritmo que hoy se ve perfectamente razonable se vuelve inutilizable con el paso de las décadas, **aunque el hardware nunca deje de mejorar** — la misma regla de la sección 2 ($k = S^{1/p}$), mirada desde el lado de quien espera que el hardware lo salve.

## Errores que hay que vigilar

- **Decir que el factor para $n^2$ es $1000/2 = 500$** (dividir $S$ entre el exponente en vez de tomar la raíz $p$-ésima de $S$). El error viene de confundir "dividir $S$ entre $p$" con "resolver $n_1^2 = S \cdot n_0^2$". La cuenta correcta es siempre despejar $n_1$ de la ecuación de costos, no operar sobre el exponente directamente.
- **Aplicar la misma fórmula de invariancia de escala ($k = S^{1/p}$) a $n \log n$**, como si $\log n$ no contara. Es un error común porque $n \log n$ "se parece" a una potencia con exponente apenas mayor que 1. La sección 3 muestra por qué eso falla: el factor para $n \log n$ **sí depende** de $n_0$, cosa que no pasa con ninguna potencia pura.
- **Reportar el factor sin su presupuesto de referencia.** El factor para $n \log n$ no es una constante universal: cambia si cambiamos $R$ (operaciones por segundo) o $T$ (el tiempo dado), porque ambos cambian $n_0$. Toda respuesta a este ejercicio debería decir explícitamente para qué $n_0$ se calculó 791, igual que toda cota mínima necesita decir en qué modelo vale (la regla general de C2 §10, que aquí se aplica aunque no estemos hablando de una cota mínima sino de una estimación).